SOFTWARE ENGINEERING PROJECT

MeetRX

BY

Phiranath Po-Ngern
Rattanan Rung-uthai
Nunthapop Nganiam

DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING
KASETSART UNIVERSITY

Academic Year 2026

MeetRX

BY

Phiranath Po-Ngern
Rattanan Rung-uthai
Nunthapop Nganiam

A Project Submitted in Partial Fulfillment of the
Requirement for the Degree of Bachelor of Engineering
(Software and Knowledge Engineering)
Department of Computer Engineering
Faculty of Engineering
KASERTSART UNIVERSITY
Academic Year 2026

Approved By:

Advisor . . . . . **Date** . . . . . .
(Asst. Prof. Dr. Hutchatai Chanlekha)

Table of Contents

List of Tables

Page

List of Figures

Page

Chapter 1

Introduction

1.1 Background

Meetings are an indispensable tool for collaborative work in both academic and professional settings. In theory, a focused meeting covering a single topic should require no more than 15–30 minutes. In practice, however, teams routinely turn short discussions into multi-hour sessions. The root cause is *topic drift*: the gradual, often unnoticed, departure of conversation from the original agenda.

The development team behind MeetRX experienced this problem first-hand during group project work. What should have been a concise weekly sync repeatedly stretched into a three-hour session because members lost track of the intended focus and the conversation wandered across unrelated topics. This observation motivated the creation of an intelligent meeting assistant that can detect drift as it occurs and help teams stay aligned.

1.2 Problem Statement

The following problems were identified through the team's own experience and stakeholder interviews:

1. **Standup meetings overrunning their allotted time.** Daily standups and progress-update meetings are intended to be short, time-boxed events. In practice, they frequently exceed their scheduled duration because participants introduce tangential topics, triggering lengthy discussions that should be handled separately.

2. **Burden on the minute-taker.** During meetings, the person responsible for taking notes cannot participate meaningfully in the discussion. Their attention is divided between listening, comprehending, and writing, which reduces both the quality of the notes and their contribution to the meeting.

3. **Repeated debates caused by poor institutional memory.** In long-term projects with frequent meetings, teams often forget what was discussed or decided in earlier sessions. As a result, the same topics are re-opened and the same decisions are re-debated, consuming time that could be spent on new work.

1.3 Solution Overview

MeetRX addresses the problems above through three integrated AI subsystems:

1. **Topic Drift Detection AI (Main Feature)**: Continuously analyses the live conversation using natural language processing techniques to assign a drift score to recent utterances. When the score exceeds a configurable threshold and the confidence is low, a large language model re-evaluation is triggered and a alert is surfaced to meeting participants, prompting them to return to the agenda. The alert will be designed to be able to configure by users.

2. **Summarisation AI**: Automatically transcribes meeting audio using OpenAI Whisper and generates a structured summary so that no human minute-taker is required. All participants can engage fully in the discussion.

3. **Agentic AI**: Provides a conversational interface that can answer questions about past meetings, look up decisions recorded in previous summaries, query the project codebase, and integrate with external tools to retrieve relevant context on demand.

1.3.1 Features

1. **Active Topic Drift Detection**: Monitors conversation flow and raises an alert when discussion deviates from the stated agenda topic.

2. **Automatic Meeting Summarisation**: Produces a concise, structured summary at the end of every session without requiring manual note-taking.

3. **Q&A Chatbot**: Allows participants to query past meeting summaries and project context using natural language.

4. **Task Assignment**: Extracts and records action items from meeting transcripts and assigns them to the appropriate team members.

1.4 Target User

MeetRX is designed for three primary user groups that often struggle with meeting or required meetings as part of their work:

- **Students**: Particularly those conducting group project meetings who need help keeping discussions on-topic and producing reliable records of decisions made.

- **Software Developers**: Engineering teams running daily standups, sprint planning, and retrospectives who want shorter, more focused meetings and automatic summaries.

- **Scrum Masters**: Facilitators responsible for sprint ceremonies who need tooling to enforce time-boxes and surface relevant historical context during discussions.

1.5 Terminology

Topic Drift A situation in which the ongoing conversation is no longer contributing to the intended meeting topic or agenda item.

Drift Score A numerical value computed by the NLP pipeline that quantifies how far the current utterance has deviated from the baseline topic embedding.

Time-Window Segment A block of transcribed utterances aggregated over a specific duration (e.g., 60 seconds) used to evaluate conversational context, reducing the noise inherent in analyzing single, isolated sentences.

Agenda A predefined list of topics that a meeting is intended to cover.

Utterance A single spoken turn or message produced by one participant during a meeting.

Summarisation The process of condensing a meeting transcript into a shorter document that captures the key points, decisions, and action items.

Agentic AI An AI system capable of autonomously invoking external tools or data sources to answer queries without explicit step-by-step instructions from the user.

LLM Large Language Model; a neural network trained on large text corpora capable of generating and understanding natural language.

Whisper An open-source automatic speech recognition model developed by OpenAI.

Chapter 2

Literature Review and Related Work

2.1 Competitor Analysis

Table 2.1: Competitive Analysis Table

Name	Meeting Summary	Q&A Chatbot	Task Assignment	Topic Drift Detection
Otter AI	✓	✓	✗	✗
Fireflies AI	✓	✓	✓	✗
Spinach AI	✓	✓	✓	✗
MeetRX	✓	✓	✓	✓

Otter AI provides transcription, automatic meeting summaries, and a Q&A interface for querying transcripts. However, it has no task assignment capability and no mechanism for detecting or alerting users to topic drift.

Fireflies AI supports transcription, summarisation, and a chatbot interface. It has task assignment feature with tools integration. Topic drift detection is absent.

Spinach AI is targeted at agile teams and offers summaries, Q&A, and task assignment with tool integration. Like the others, it provides no topic drift detection.

MeetRX is the only solution in this comparison that addresses all four dimensions, and is the only product that actively detects and alerts participants to topic drift during a live meeting.

2.2 Literature Review

This section reviews academic research and technical reports relevant to the design and implementation of MeetRX, focusing on four key areas: transcription, meeting summarization, task allocation, topic drift detection, and agentic AI systems. These works provide the technical foundation for the design of MeetRX and how it aids in selecting appropriate metrics for evaluating the system's performance.

2.2.1 Speech Analysis

Speech analysis is a fundamental component of the system, enabling the automatic conversion of meeting audio into text. The study in[1] explores the integration of NLP with VoIP systems to achieve low-latency transcription.

Another work[2] propose to use Recurrent Neural Network Transducers models to improve accuracy and speed of transcription in noisy environments, which is crucial for real-world meeting scenarios.

To evaluate the performance of the transcription module, we will use metrics such as Word Error Rate (WER)[3] and latency. WER measures the accuracy of the transcription by comparing the transcribed text to a reference transcript, while latency assesses the time taken from audio input to text output.

2.2.2 Meeting Summarization

Meeting summarization aims to shorten and condense the content of a meeting transcript while preserving key information. The MISeD dataset[4] introduce a method for generating dialogue data, particularly for meeting scenarios, the QMSum dataset[5] provides annotated meeting transcripts designed for query-based summarization tasks.

Evaluating summarization quality can be done using ROUGE[6] and BERTScore[7], which measure the overlap of n-grams and semantic similarity between the generated summary and reference summaries.

2.2.3 Task Allocation

Task allocation is a critical component of team productivity in agile environments. The study in[8] explores the use of machine learning algorithms to assign tasks in distributed software development teams based on historical performance and expertise.

Similarly, the TaskAllocator framework[9] proposes a recommendation-based approach that assigns tasks according to roles and past project data.

These approaches highlight the potential of data-driven task assignment to reduce manual coordination and improve efficiency. By learning from historical patterns, systems can make informed decisions about task distribution from the available data.

Inspired by these works, MeetRX decides to implement a task assignment feature that extracts action items from meeting transcripts and assigns them to appropriate team members based on expertise or roles.

2.2.4 Topic Drift Detection

Topic drift detection is the core feature of MeetRX, enabling active alerts when a meeting conversation deviates from the agenda. Understanding the temporal nature of human attention is critical for establishing when this drift typically occurs and defining the system's detection windows. Foundational studies on attention spans note that active engagement reliably drops after 15 to 20 minutes without a shift in stimulus[10]. Furthermore, industry research by the Microsoft Human Factors Lab[11] demonstrated that cognitive fatigue begins to accumulate significantly after 30 to 40 minutes of sustained meeting time, naturally increasing the likelihood of conversational tangents. When a meeting does drift, research by Mark et al.[12] highlights that it takes an average of 23 minutes for a professional to fully regain deep cognitive focus. Consequently, MeetRX utilizes a 550-second (approximately 9-minute) overarching rolling window to catch the early onset of cognitive fatigue before it spirals into a costly distraction.

The technical foundation for segmenting these shifts dates back to the **TextTiling Algorithm**[13], which utilizes a sliding window approach to compute lexical similarity between adjacent blocks of text to identify subtopic boundaries. Modern implementations, such as the **TIAGE framework**[14], evolve this concept by using transformer-based embeddings and **cosine similarity** to detect changes in dialog. By comparing the vector representation of the live transcript window against the predefined agenda, systems can quantify 'drift' more accurately than through keyword matching alone.

Another study[15] emphasizes that maintaining context is crucial in mixed-initiative conversations to prevent false positives in drift alerts. Based on these combined cognitive and algorithmic findings, MeetRX implements a sliding window architecture utilizing a 50-second base segment advancing with a 25-second stride. The NLP model continuously computes a drift confidence score for each 50-second block against the predefined agenda. High-confidence drifts are flagged immediately, while low-confidence blocks are forwarded to an LLM for secondary validation. To prevent false alarms triggered by acceptable social tangents, the system aggregates these flagged micro-segments into the overarching 550-second rolling context. If the volume of flagged windows across this 9-minute span exceeds a configurable threshold, the system issues a subtle alert to realign the meeting.

2.2.5 Agentic AI and Two-Tier System Design

Agentic AI allow systems to perform complex tasks automatically by integrating LLMs with toolchains[16]. However, the high computational cost and latency of large-scale models pose challenges for live-interactive systems.

To address this, MeetRX adopts a **Two-Tier Architecture** inspired by **LLM Cascade Systems** and **FrugalGPT**[17]. This approach utilizes a 'cascade routing' strategy:

1. **First Tier (NLP Based Model):** A lightweight model or traditional NLP similarity check (like the cosine similarity confidence check) processes the initial input to determine if a drift has occurred or if a

query can be handled locally. If the confidence is high, the system responds directly without invoking the LLM, thus saving time and computational resources.

2. **Second Tier (LLM):** Only when a the confidence is not high enough, or a specific 'agentic' action is required, is the request pushed to a more powerful LLM.

This cascading mechanism, as explored by Stanford researchers[17], significantly reduces API costs and improves response latency while maintaining high accuracy. This ensures MeetRX would save computational resources more than using pure LLMs.

2.2.6 Agentic AI and Code Analysis

Recent advancement in agentic AI enables systems to perform complex tasks autonomously. Frameworks such as those described in[16] demonstrate how LLMs can be integrated with toolchains to analyze code repositories and provide insights on the user queries.

In addition, the agentic AI system for the platform such as GitHub's Agentic Workflows[18] and agentic code review systems[19] show how AI can assist in automating software development tasks, including code analysis and review.

Evaluating the performance of agentic AI systems involves metrics such as task success rate, response accuracy, and tool usage efficiency[20]. These metrics ensure that the system produces reliable and useful outputs.

Based on these findings, MeetRX integrates an agentic AI component that allows users to query past meeting information to retrieve relevant details.

2.3 Summary

This chapter has reviewed the competitive landscape of meeting productivity tools and the relevant academic research that informs the design of MeetRX. The analysis highlights the unique features of MeetRX, particularly its active topic drift detection, which sets it apart from existing

solutions. The literature review provides a solid foundation for the technical implementation of MeetRX and guides the selection of appropriate evaluation metrics for assessing its performance in subsequent chapters.

Chapter 3

Requirement Analysis

3.1 Stakeholder Analysis

1. **Student**: University students working in group projects need a tool that detects when their meeting drifts off the project agenda and automatically generates meeting minutes so that all members can participate in the discussion rather than taking turns as note-taker.

2. **Software Developers (Office Workers)**: Software engineers attending daily standups and sprint ceremonies benefit from automatic meeting summaries and from an agent that can surface relevant decisions from past sessions, reducing repeated debate.

3. **Scrum Masters / Project Managers**: Facilitators need visibility into whether a meeting is staying on-topic and access to a historical record of decisions so they can run tighter, more accountable ceremonies.

3.2 User Stories and Use Cases

3.2.1 Student

1. As a **student**, I want the system to alert me when my group's conversation goes off-topic, so that we can finish our meeting within the scheduled time.

2. As a **student**, I want an automatic summary of each meeting, so that I do not have to assign a dedicated note-taker.

3. As a **student**, I want to query what was decided in a previous meeting, so that I do not have to re-read full transcripts.

3.2.2 Software Developer

1. As a **software developer**, I want standup meetings to stay focused on project progress, so that they finish within 15 minutes.
2. As a **software developer**, I want action items extracted automatically from the meeting transcript and assigned to the correct team member, so that nothing falls through the cracks.
3. As a **software developer**, I want to ask the AI agent about a past architectural decision, so that I do not have to search through old chat logs or documents.

3.2.3 Scrum Master

1. As a **scrum master**, I want a active indicator that shows whether the current discussion is on-topic, so that I can redirect the team without interrupting the conversation flow.
2. As a **scrum master**, I want all sprint summaries stored and indexed, so that I can reference them during retrospectives.

3.3 Use Case Diagrams

The use case diagram below shows that the system has two main actors: Meeting Participant and Project Manager/Leader or Scrum Master. The Meeting Participant can receive topic drift alerts, query past meetings, and view meeting summaries. The Project Manager/Leader or Scrum Master can start meetings using MeetRX bot, view meeting summaries, and manage tasks. Both actors interact with the system to ensure that meetings are productive and that important information is easily accessible.

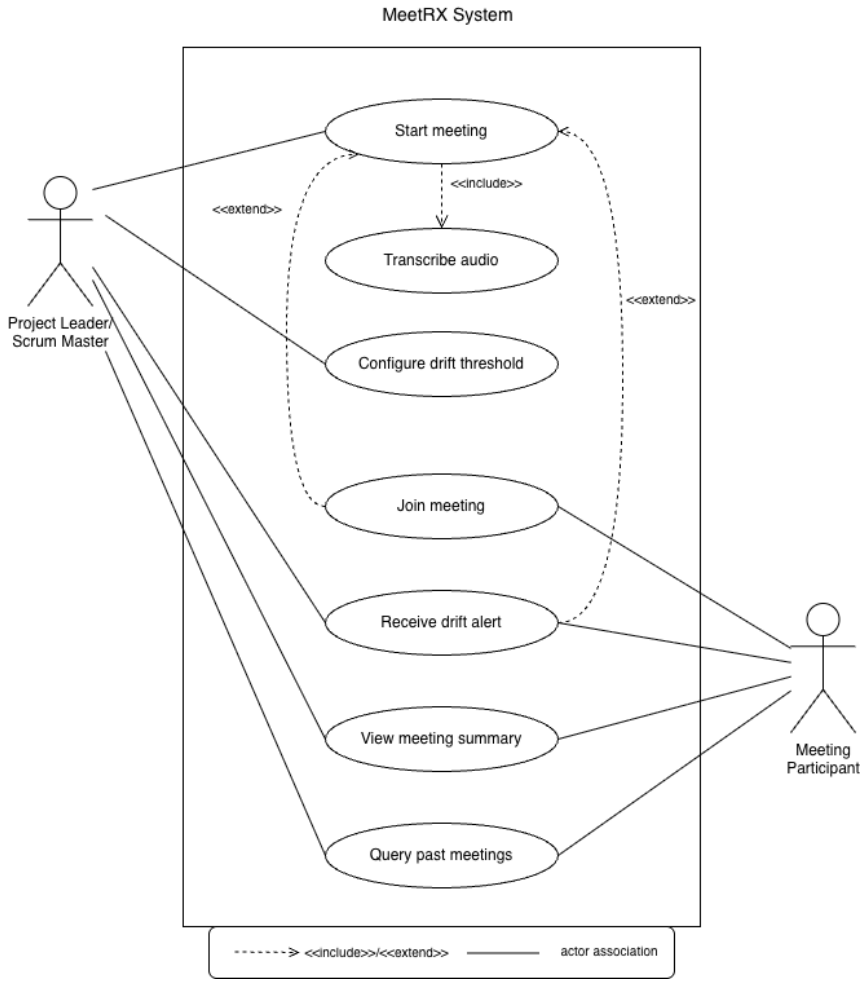

Figure 3.1: Use Case Diagram

3.4 Use Case Model

UC-01: Start Meeting

Actor	Meeting Participant
Precondition	The participant is authenticated and has a meeting agenda prepared.
Main Flow	1. Participant opens the MeetRX dashboard. 2. Participant clicks “Start Meeting” and enters agenda topics. 3. System begins audio capture and topic embedding initialisation. 4. Meeting session is created and all participants are notified.
Postcondition	An active meeting session exists with an initialised topic baseline.

UC-02: Receive Topic Drift Alert

Actor	Meeting Participant
Precondition	A meeting session is active.
Main Flow	1. System segments live transcription into 50-second time-windows with a 25-second stride. 2. NLP module computes a confidence score for the current 50-second window. 3. If high confidence, the window is flagged as "drifted." 4. If low confidence, the window is evaluated by the LLM; if the LLM confirms, it is flagged. 5. System tracks the total count of flagged windows over a rolling 550-second period. 6. If the count of flagged windows exceeds the maximum threshold, the system logs the event. 7. System displays a subtle visual drift alert (e.g., pulsing agenda item) visible to participants. 8. Participant acknowledges the alert and guides conversation back to the agenda.
Alternative Flow	If the participant dismisses the alert, the system logs the dismissal, resets the flagged window counter, and resumes monitoring.
Postcondition	Alert is logged; meeting continues.

UC-03: Query Past Meetings

Actor	Meeting Participant
Precondition	At least one completed meeting summary exists in the system.
Main Flow	1. Participant opens the Q&A interface. 2. Participant types a natural language question. 3. Agentic AI retrieves relevant summaries and generates an answer. 4. System displays the answer with source meeting references.
Postcondition	Participant receives an answer grounded in historical meeting data.

3.5 User Interface Design

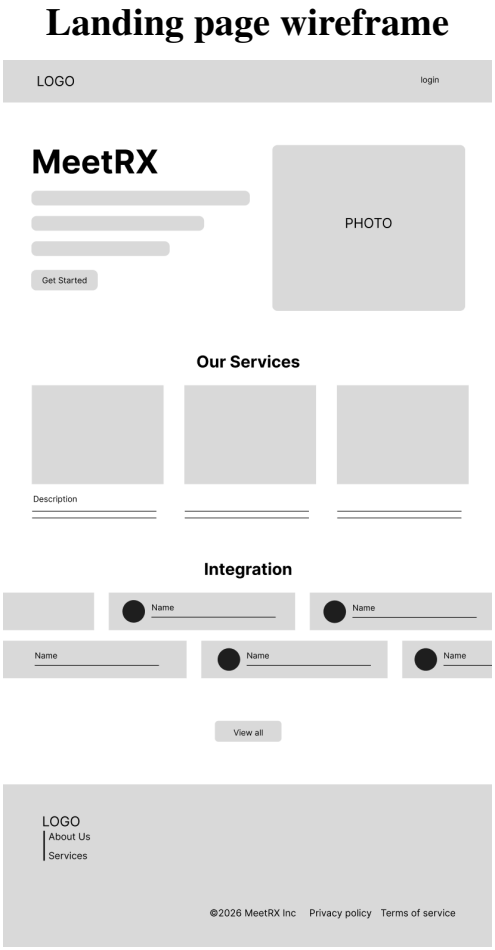

Figure 3.2: Landing Page Wireframe

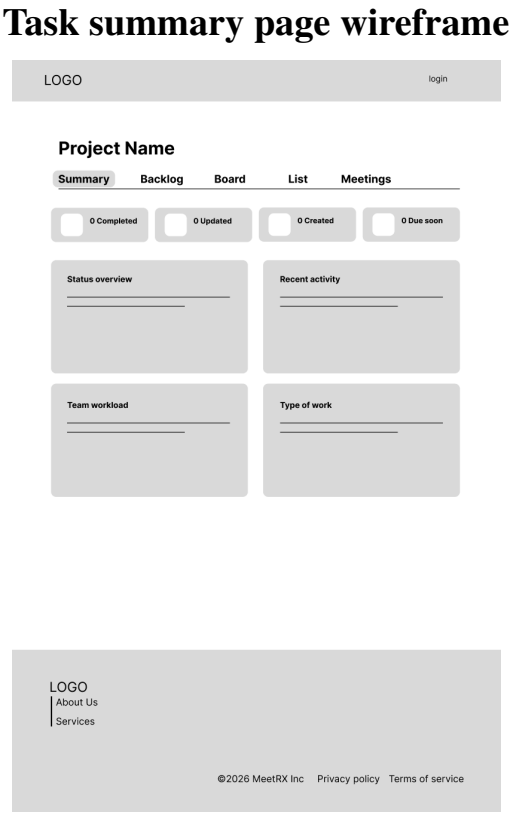

Figure 3.3: Task Summary Page Wireframe

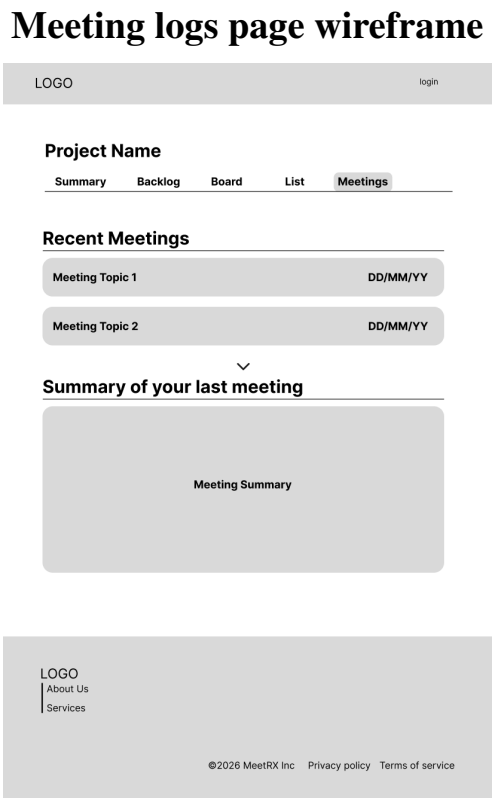

Figure 3.4: Meeting Logs Page Wireframe

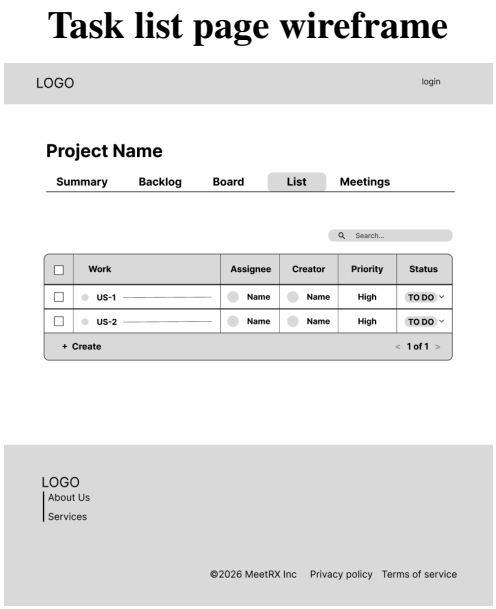

Figure 3.5: Task List Page Wireframe

Back logs page wireframe

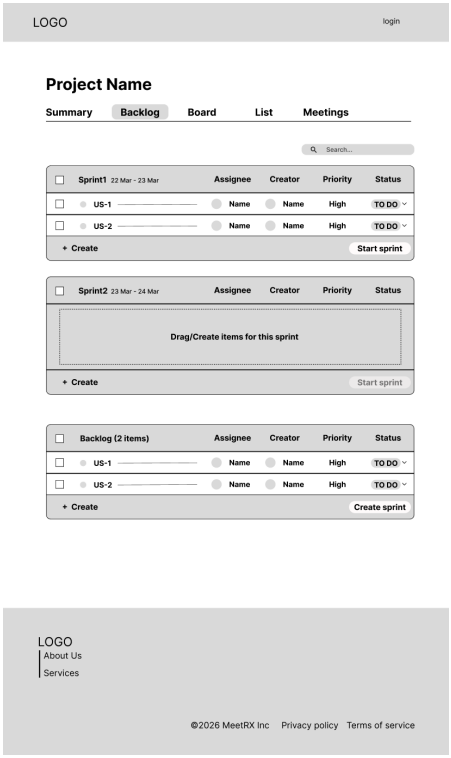

Figure 3.6: Back Logs Page Wireframe

Chapter 4

Software Architecture Design

4.1 System Architecture Overview

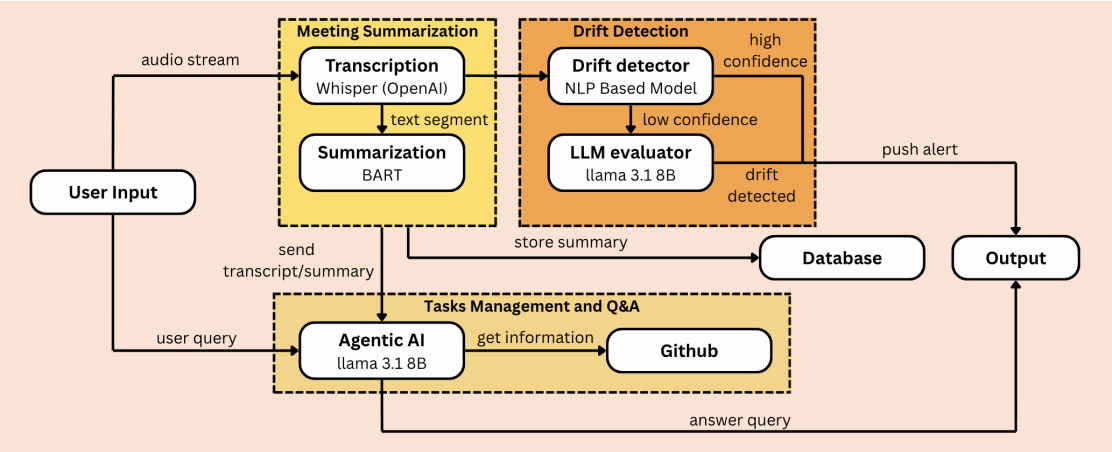

Figure 4.1: System Architecture

The system begins with user input in the form of audio or queries. Audio streams are processed by the transcription module to convert speech into text. For user queries, the agentic AI component retrieves stored data and external resources (e.g., GitHub) to generate responses. At the end of the user's meeting, the system outputs the meeting's summaries, and action items brought up in the meeting for further task managements.

4.2 Software Design

The system operates as a continuous pipeline that processes the user meeting's audio input and supports task management and query answering.

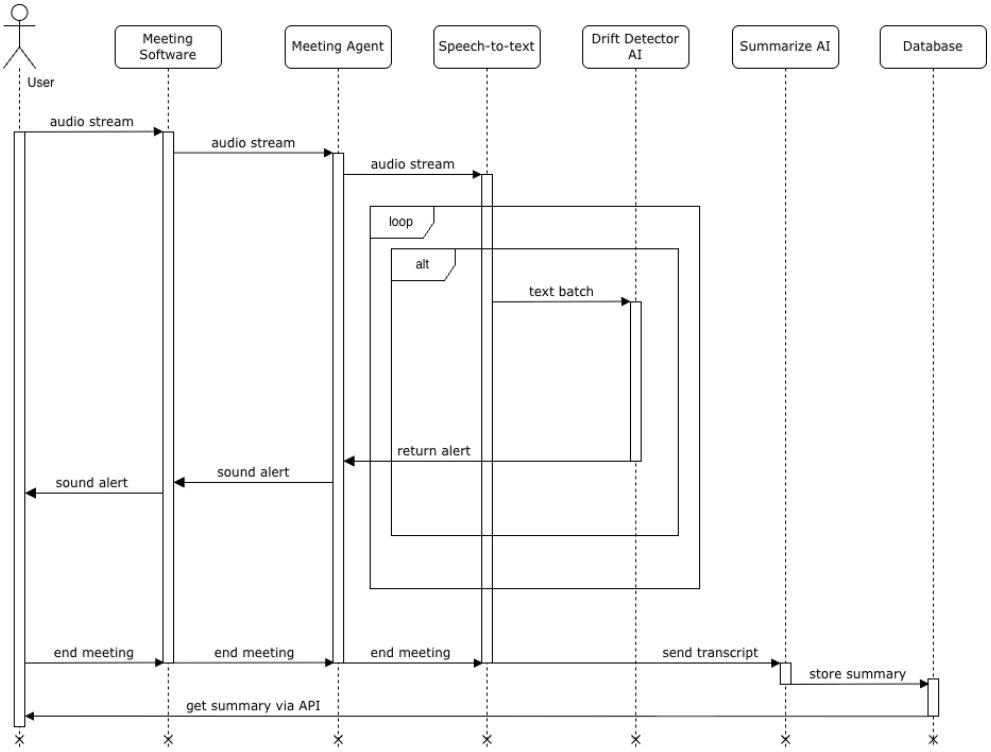

Figure 4.2: Sequence Diagram of Topic drift detection and summarization

When the user add our AI into the meeting, it records the meeting's audio and sends it to the transcription module, which converts speech into text segments using Whisper. The texts are then passed to the topic drift detection module every few minutes, where an NLP-based model combined with an LLM Evaluator analyzes the content to identify any deviations from the main topic and notify the user if a drift is detected.

After the meeting, the transcript is passed to the summarization module, which generates a concise summary of the meeting content. Both the transcript and the summarized information are then sent to the agentic AI module, which handles task management and question answering. This module can also retrieve additional context from external sources such as GitHub. When a user submits a query, it is processed by the agentic AI, which generates a response using both stored data and external information. Transcripts and summaries are stored in the database for later retrieval.

Finally, all outputs, including summaries, alerts, and query responses, are delivered to the user through the output interface shortly after processing.

4.3 AI and Data Design

4.3.1 Data Sources and Collection

This project will utilize publicly available datasets to train and evaluate the AI components including topic drift detection, speech-to-text transcription, question answering, text summarization, and task management. For each topic, we will use the dataset below:

- **AMI Meeting Corpus**: Provides over 170 annotated meeting recordings along with transcripts, allowing the system to learn realistic meeting conversations and structures.
- **TIAGE dataset**: Used for topic drift detection, containing approximately 500 text samples focused on conversational topic classification.
- **TaskAllocator**: The dataset contains more than 600 entries of project names, tasks, descriptions, and roles, which we will use to train our task management model to assign tasks to the user's team.

4.3.2 Model Design

The system adopts a modular approach by integrating multiple AI models, each designed for a specific task within the meeting workflow.

- **Drift Detection**: A combination of an NLP-based model and an LLM Evaluator is utilized via a sliding-window algorithm with two-step validation. The live transcription stream is chunked into 50-second Time-Window Segments, advancing with a 25-second stride to capture continuous context.

  For each 50-second segment, the system employs cascade validation:

  - **Step 1 (NLP Filter):** The NLP model computes a drift confidence score against the agenda. If the score indicates high confidence of drift, the 50-second window is immediately flagged as "drifted."
  - **Step 2 (LLM Confirmation):** If the NLP score indicates low or uncertain confidence, the 50-second transcript is forwarded to the **LLaMA 3.2 1B** LLM. The LLM performs a context-aware analysis to determine if the window should be flagged as "drifted."

  To prevent false alarms triggered by acceptable social tangents, these individual window flags are aggregated over a broader 550-second (9-minute) rolling context. An active alert (a subtle visual nudge) is only triggered to the users if the total number of flagged 50-second windows within this 550-second span exceeds a configurable threshold.

- **Speech-to-text Transcription**: A **Whisper** is used due to its strong performance in live transcription and multilingual support. The model will convert spoken audio into text, which serves as the foundation for further processing.
- **Question Answering**: A **BERT-based model (BertQA)** is used, because this model is effective at extracting precise answers from contextual data. It is integrated into an agentic system that retrieves relevant information from past meetings and generates context-aware responses.
- **Text Summarization**: The system will utilize **BART** for its strong performance in generating concise and meaningful summaries from long meeting transcripts.
- **Task Management**: The **LLaMA 3.2 1B** or equivalent models will be used to identify and extract actionable tasks from meeting content. This model can detect responsibilities, deadlines, and action items from natural language.

All models are integrated into a pipeline where outputs from one component are used as inputs for others. For example, transcription feeds

into summarization and question answering. Prompt engineering techniques are applied when interacting with large language models to ensure consistent and relevant outputs. The selected models are chosen based on their effectiveness in natural language processing tasks and their ability to handle live conversational data.

4.3.3 Evaluation

The performance of the AI components is evaluated using task-specific metrics.

Topic Drift Detection: Precision, Recall, and F1-score are used to evaluate the model's ability to correctly identify whether a conversation has deviated from the main topic. Precision measures the correctness of detected drifts, while recall evaluates how many actual drifts are successfully identified. The F1-score provides a balanced measure of both.

Speech-to-Text Transcription: Word Error Rate (WER) is used to assess transcription accuracy by comparing the generated transcript with the ground truth. It reflects the proportion of errors in the transcription output.

Question Answering: Evaluation focuses on task success rate, the number of steps required to reach a solution, and human evaluation. These metrics assess whether the system effectively achieves the user's goal and provides useful responses.

Text Summarization: ROUGE metrics, including ROUGE-N and ROUGE-L, are used to evaluate the quality of generated summaries. These metrics measure the overlap and similarity between the generated summary and reference summaries.

Task Management: Standard classification metrics such as accuracy, precision, recall, and F1-score are used to evaluate the system's ability to correctly extract and manage tasks. These metrics ensure consistent performance across different scenarios.

Together, these evaluation metrics provide a comprehensive assessment of the system's performance across all AI components.

Chapter 5

Software Development

5.1 Schedule

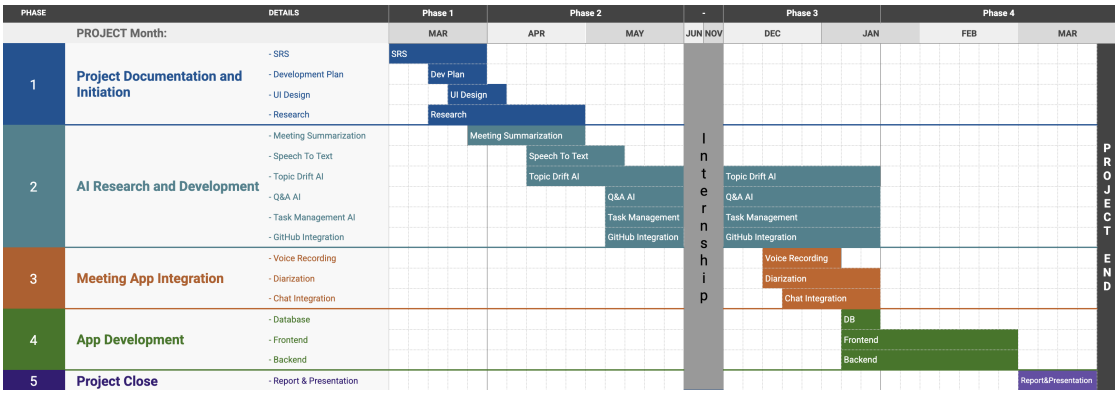

Figure 5.1: Project Schedule

The project is organized into five phases spanning from March to the following March. The schedule follows the Scrum methodology with two-week sprints, allowing for iterative development and continuous feedback throughout the project lifecycle.

Phase 1: Project Documentation and Initiation (March – May) focuses on establishing the project foundation. The team begins by writing the Software Requirements Specification (SRS), covering background, problem statement, solution overview, stakeholder analysis, user stories, use case diagrams, domain model, system architecture, and AI design. Concurrently, a development plan is produced that defines the Gantt chart, team responsibilities, and sprint structure. UI wireframes and mockups for the Meeting Dashboard, Summary View, and Q&A Interface are designed in Figma and validated against user stories. In parallel, a

literature review is conducted covering topic drift detection methods (cosine similarity, LDA, Sentence-BERT), meeting summarisation models, and agentic AI frameworks (ReAct) to select the most appropriate technologies for the system.

Phase 2: AI Research and Development (April – December) is the core development phase. Meeting Summarisation is implemented first using OpenAI GPT-4o with a structured prompt designed to produce an overview paragraph, a key decisions list, and an action items table. Speech-to-text transcription is then integrated using WhisperX, with audio chunk streaming, speaker turn segmentation, and utterance storage in PostgreSQL. The Topic Drift AI is built using Sentence-BERT embeddings and cosine similarity scoring, with an LLM Evaluator fallback that calls GPT-4o to confirm drift when the score exceeds a configurable threshold. The Q&A AI and Task Management AI are implemented as part of the Agentic AI module using a ReAct loop, integrating semantic search over ChromaDB and structured queries against PostgreSQL for action item retrieval and management. Finally, the GitHub Integration tool is added to the Agentic AI, enabling codebase context retrieval via the GitHub API based on user queries.

Phase 3: Meeting App Integration (December – January) focuses on connecting the AI components to a real meeting environment. Voice recording is implemented in the frontend using the Web Audio API, streaming audio chunks to the API Gateway via WebSocket with handling for browser permissions and network reconnection. A speaker diarization model is integrated to attribute each utterance to a specific participant, producing speaker-labeled transcript segments for more accurate task assignment. Finally, MeetRX is integrated with third-party meeting platforms such as Google Meet and Microsoft Teams via their respective APIs or browser extensions.

Phase 4: App Development (January – March) covers the full implementation of the application. The database layer is set up, including the PostgreSQL schema for users, sessions, transcripts, summaries, and

action items, as well as ChromaDB for vector storage and Redis for session caching and pub/sub event streaming. The React and TypeScript frontend is implemented with the Meeting Dashboard, Summary View, and Q&A Interface. The FastAPI backend is completed with all REST and WebSocket endpoints, service orchestration, authentication, and a CI/CD pipeline using GitHub Actions, targeting a minimum of 80% unit test coverage across all backend modules.

Phase 5: Project Close (March) concludes the project. The team finalises the SRS document, prepares the presentation slides, records a system demonstration video, and submits all deliverables. A final retrospective is conducted to document known limitations and directions for future work.

Reference

Bibliography

[1] K. Rajchandar, P. N. Hegde, U. Dwivedi, K. Chouhan, K. S. Sidhu, and R. Velagala, "Natural language processing for real-time transcription in voip systems," in *Proceedings of the 2025 International Conference on Computing Technologies and Data Communication (ICCTDC)*, 2025, pp. 1–5.

[2] J. Ma and Y. Liu, "Research on real-time speech transcription optimization based on recurrent neural network transducer," in *Proceedings of the 2023 IEEE International Conference on Image Processing and Computer Applications (ICIPCA)*, 2023, pp. 1571–1576.

[3] T. P. Code, "Calculate word error rate (wer) in python," 2023. [Online]. Available: https://thepythoncode.com/article/calculate-word-error-rate-in-python

[4] L. Golany, F. Galgani, M. Mamo, N. Parasol, O. Vandsburger, N. Bar, and I. Dagan, "Efficient data generation for source-grounded information-seeking dialogs: A use case for meeting transcripts," *arXiv preprint arXiv:2405.01121*, 2024.

[5] H. Zhong, C. Xiao, L. Tang, Z. Lei, T. Yang, and J. T. Kwok, "Qmsum: A new benchmark for query-based multi-domain meeting summarization," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. ACL, 2021, pp. 349–365.

[6] Santhosh, "Understanding bleu and rouge score for nlp evaluation," 2023. [Online]. Available: https://medium.com/@sthanikamsanthosh1994/understanding-bleu-and-rouge-score-for-nlp-evaluation

[7] Abonia, “Bertscore explained in 5 minutes,” 2023. [Online]. Available: https://medium.com/@abonia/bertscore-explained-in-5-minutes

[8] D. Al-Fraihat, Y. Sharrab, A.-R. Al-Ghuwairi, H. Alzabut, M. Beshara, and A. Algarni, “Utilizing machine learning algorithms for task allocation in distributed agile software development,” *PMC*, 2024. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC11567032/

[9] S. Shafiq, A. Mashkoor, C. Mayr-Dorn, and A. Egyed, “Taskallocator: A recommendation approach for role-based tasks allocation in agile software development,” *arXiv preprint arXiv:2103.02330*, 2021.

[10] N. A. Bradbury, “Attention span during lectures: 8 seconds, 10 minutes, or more?” *Advances in Physiology education*, vol. 40, no. 4, pp. 509–513, 2016.

[11] Microsoft Human Factors Lab, “Research proves your brain needs breaks,” Microsoft WorkLab: Work Trend Index, Tech. Rep., 2021. [Online]. Available: https://www.microsoft.com/en-us/worklab/work-trend-index/brain-research

[12] G. Mark, D. Gudith, and U. Klocke, “The cost of interrupted work: more speed and stress,” *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*, pp. 107–110, 2008.

[13] M. A. Hearst, “Texttiling: Segmenting text into multi-paragraph subtopic passages,” *Computational Linguistics*, vol. 23, no. 4, pp. 33–64, 1997.

[14] H. Xie, Z. Liu, C. Xiong, Z. Liu, and A. Copestake, “Tiage: Topic-aware dialogue generation,” *arXiv preprint arXiv:2109.04562*, 2021.

[15] R. Konigari, S. Chand, V. V. Alluri, and M. Shrivastava, “Topic shift detection for mixed initiative response,” in *Proceedings of the 22nd Annual Meeting of the Special Interest Group on Discourse and Dialogue*, 2021, pp. 189–195.

[16] ActiveWizards, "Build an ai agent for github code analysis with langchain," 2024. [Online]. Available: https://activewizards.com/blog/build-an-ai-agent-for-github-code-analysis-with-langchain

[17] L. Chen, M. Zaharia, and J. Zou, "Frugalgpt: How to use large language models while reducing cost and latency," *arXiv preprint arXiv:2305.05176*, 2023.

[18] GitHub, "Github agentic workflows," 2024. [Online]. Available: https://github.github.com/gh-aw/

[19] CyberArk, "Agentic code review demo," 2024. [Online]. Available: https://github.com/cyberark/agentic-code-review-demo

[20] GPTalk, "How to evaluate agentic ai: Key metrics and real-world examples," 2024. [Online]. Available: https://medium.com/gptalk/how-do-you-evaluate-agentic-ai-key-metrics-real-world-examples-781004de0280